TeleConnect

Technical Interview Q&A

Complete answers to expected technical questions

Part 1 -- Data & Churn Model

1.1 Data Cleaning

Q: Walk us through the data quality issues you found and how you handled them.

The raw dataset (5,050 rows) had 13 distinct issues. I built a centralized cleaning function (src/data_cleaning.py) that: (1) drops 50 exact duplicate rows; (2) normalizes categorical encodings (M->Male, F->Female, BT->Bank transfer, fiber->Fiber optic); (3) handles out-of-range sentinels (satisfaction_score > 10, age > 100, tenure < 0 -> NaN); (4) fixes a decimal-shift error in total_charges (rows off by 100x vs tenure*monthly get divided by 100); (5) splits internet_service 'No' from missing values (different imputation). Result: 5,000 rows passed forward. I documented before/after counts for each issue -- that transparency was part of the deliverable.

Q: Why did you use median imputation instead of forward-fill or mean?

Median is robust to outliers and doesn't assume temporal ordering (we have cross-sectional customer data, not time series). For salary/charges/tenure, median reflects the 'typical' case better than mean when you have long tails or errors. It also avoids assumptions about relationships between numeric fields. For categorical, I used most_frequent mode (also robust). I avoided SMOTE upsampling because it synthesizes implausible rows in mixed continuous+one-hot spaces and risks data leakage if not applied inside CV.

Q: Your cleaning code is in src/data_cleaning.py. Why not just do it in the notebook?

Because the churn model inference code (src/predict.py) must run the exact same transformations as training, or you get train/serve skew. A customer's raw data in production goes through data_cleaning.py -> features.py -> pipeline -- same code as training. If I only cleaned in the notebook, predict.py would have to duplicate all that logic, and they'd diverge. Sharing code ensures consistency and makes the model trustworthy.

1.2 Feature Engineering & EDA

Q: Tell us about the features you engineered.

I added 5 engineered features: (1) charges_per_tenure_month = monthly_charges / max(tenure_months, 1), capturing billing intensity over time; (2) tickets_per_tenure_year = annual support ticket rate, a friction signal; (3) expected_vs_actual_charges_gap = |total_charges - tenure*monthly|, catches billing anomalies; (4) tenure_bucket = [<6m, 6-24m, 24m+], capturing non-monotonic early-churn risk; (5) has_bundle = 1 if phone+internet both exist, 0 else, a stickiness signal. Each was motivated by domain intuition (what would a telecom ops person care about?) and validated in EDA correlations.

Q: What did your EDA show? What were the top churn drivers?

Contract type was the overwhelming #1 signal: month-to-month customers churn at 42%, one-year at 11%, two-year at 2% (point-biserial rho=0.54). Tenure had non-linear effect (early customers churn more than middle, then stabilize) -- captured by tenure_bucket. Satisfaction inversely correlated with churn (spearman=-0.38). Customers with 0 add-on services churned more. Support tickets also predicted churn (customers who call support are less sticky). I used point-biserial for continuous vs binary, Cramer's V for categorical vs binary, and mutual information as a unified ranker across types. All three agreed on the top 5.

1.3 Model Selection & Training

Q: You trained both Logistic Regression and XGBoost. Why Logistic Regression won?

Both scored well on ROC-AUC (0.71 vs 0.70), but Logistic Regression won on PR-AUC (0.534 vs 0.527) -- the metric that matters most for imbalanced churn. The reason: on this dataset, the churn signal is largely additive. Contract type + tenure + satisfaction explain most of the variance; there are few interaction effects that XGBoost would exploit. Logistic Regression is also more interpretable (coefficients are readable), requires less tuning, and is lower-risk for deployment. I'd revisit XGBoost if we had 10x more data or richer features. The choice reflects a bias toward simplicity + interpretability, which is right for a production retention system.

Q: How did you handle class imbalance (36% churn)?

I used class_weight='balanced' in LogisticRegression, which auto-adjusts loss weights by class frequency (up-weights the minority). For XGBoost, I set scale_pos_weight=neg/pos (1.74). I rejected SMOTE because: (1) it synthesizes rows in a mixed continuous+one-hot space, creating implausible rows; (2) it can leak information if applied outside cross-validation; (3) reweighting is simpler and preserves the original feature distribution. I measured recall (74% of churners caught at threshold 0.5) and PR-AUC as the outcome metrics.

Q: What are the top 3 limitations of your model?

1) No temporal validation -- I use a random train/test split. Real churn prediction should use time-based split (train on Jan-Jun, test on Jul-Dec) to catch drift. 2) Threshold fixed at 0.5 -- in production, the operating point should be tuned to offer cost vs customer lifetime value (CLV), which I didn't have. 3) No probability calibration -- I didn't apply Platt scaling or isotonic regression, so confidence scores may be overconfident. These are all addressable with more data / domain input.

Part 2 -- Agent & Orchestration

2.1 Tool Design

Q: You have 5 tools. Why these 5? Can you add a 6th?

The 5 tools cover the rep's workflow: lookup (get customer data) -> predict (risk score) -> offers (retention options) -> log (record outcome) -> escalate (hand off complex cases). They form a natural chain that the LLM learns from the system prompt. Adding a 6th is trivial -- I designed the system around a TOOL_REGISTRY dict. A new tool = one schema + one implementation function + one dict entry. The orchestrator (src/agent/orchestrator.py) loops over the registry; no rewrite needed. I could add, say, 'check_payment_history' or 'get_competitor_offers' with zero changes to the orchestrator.

Q: Walk us through a multi-step interaction. What does the agent do step-by-step?

Rep: 'Customer TC-001096 might leave.' Agent: (1) calls lookup_customer(TC-001096), gets profile + _features dict; (2) extracts _features, calls predict_churn({customer_data}), gets {prob=0.76, tier='high', factors=[...]}; (3) extracts tier='high', calls get_retention_offers(high, 'Month-to-month'), gets 3 eligible offers; (4) synthesizes final text: 'High risk (76%), recommend 20% Loyalty Discount, here's why...'; (5) calls log_interaction with the outcome. The loop repeats only if the agent returns a tool_use block (stop_reason='tool_use'); when it returns text (stop_reason='end_turn'), the conversation ends. Safety max: 8 turns to prevent infinite loops.

Q: How does your system handle ambiguity (e.g. no customer ID provided)?

The system prompt says: 'If the rep describes a customer but gives no customer ID, ask for it before doing anything else. Do not invent an ID.' The LLM reads this and, when it sees a message like 'I have a risky customer on the phone,' it returns a text block (not a tool_use block) asking 'Could you give me the customer ID?' No tools are called. In mock mode, I use a regex pattern check: if no 'TC-' ID found in the message, the mock returns a clarification request. This is tested explicitly (eval case: ambiguous_no_id).

2.2 LLM Provider Abstraction

Q: You have 3 LLM providers (Anthropic, Ollama, Mock). Why?

Anthropic (Claude) is the primary real LLM. Ollama lets reviewers run the real agent locally with no API key (free GGUF models like qwen2.5). Mock lets the entire system -- eval suite, Streamlit app, CI/CD -- run end-to-end with zero network calls. Each is behind a thin abstraction (llm_client.py) that normalizes to Anthropic's content-block format. This lets me swap providers by env var (LLM_PROVIDER=anthropic|ollama|mock). The trade-off: the abstraction means some provider-specific features (e.g. streaming, vision) are not exposed. But for a retention chatbot, that's fine.

Q: The mock LLM -- how does it work? Isn't it fake?

The mock is deterministic, not fake. It uses pattern matching (regex for customer IDs, keyword lists for escalation triggers) to decide which tool to call next. It correctly chains lookup -> predict -> offers -> synthesize. It handles ambiguity (no ID -> ask), escalation (legal threat -> escalate), and out-of-scope (weather -> decline, no tools). What it can't do: reason about conflicting signals ('model says low but profile looks bad'). That requires a real LLM. The

mock is labeled everywhere (README, Streamlit sidebar, scorecard banner) so reviewers know what they're seeing. It's perfect for demos and evals; for production, a real LLM is required.

2.3 Orchestration Loop

Q: Explain the orchestration loop. What happens each turn?

Each turn: (1) append the rep's message to the messages list; (2) send [system_prompt, messages, tool_schemas] to the LLM; (3) LLM returns content blocks (text, tool_use); (4) for each tool_use: look up tool in TOOL_REGISTRY, call it, record the result (tool name, order, latency_ms); (5) append tool results as tool_result blocks back into messages; (6) loop. When LLM returns only text blocks (stop_reason='end_turn'), exit and return the agent's final response + full trace. Max 8 turns as a safety guard. The trace (tool order, args, returns) is consumed by the Streamlit app (renders timeline) and eval harness (computes accuracy).

Part 3 -- Evaluation & Testing

3.1 Test Suite & Metrics

Q: You have 14 test cases across 7 categories. Why these cases?

The brief asked for >=12 cases covering: single-tool, multi-step chaining, ambiguous input, out-of-scope, escalation, model-disagreement, adversarial. I designed 14 cases to thoroughly cover each category with realistic scenarios. Single-tool tests whether the agent stops after one tool when appropriate. Multi-step tests the full chain. Ambiguous tests missing-ID handling. Escalation tests legal/regulatory threats. Model-disagreement tests whether the agent flags low model scores with bad profile signals. Adversarial tests edge cases (prompt injection, unknown IDs, conflicting instructions). Each case has: user_input, expected_tools (name/order/params), must_not_call tools, quality_criteria, and response substrings to check.

Q: You use 3 automated metrics. Explain why these 3?

Tool selection accuracy (combination of Jaccard + ordered correctness) directly measures what the brief cares about: does the agent call the right tools in the right order? Parameter extraction accuracy checks whether key arguments (e.g., correct customer_id) were passed correctly. Response completeness checks whether the final text contains required concepts (e.g., 'high risk', 'offer', not found'). These three cover the full pipeline: right tools, right params, good synthesis. All three are deterministic (no LLM), run in milliseconds, and are cross-checks against the LLM judge.

Q: Tell us about the LLM-as-judge. How does it avoid bias?

The judge is a separate LLM model (claude-opus-4-8 vs agent's claude-sonnet-4-6) for independence. It scores on 4 dimensions (factual_correctness, tool_use_appropriateness, actionability, hallucination), each with anchored 1/3/5 scores (not vague 'good/bad'). For example, factual_correctness: score 1='contradicts tool results', 3='minor mismatch', 5='fully grounded'. Temperature=0 (deterministic). Limitations: positivity bias (LLMs tend to score high), prompt sensitivity. Mitigations: explicit anchors, different model, automated metrics as cross-checks. For production, I'd recommend periodic hand-labeling to measure Cohen's kappa.

Q: What does your scorecard show? Pass rate?

In mock mode (current demo): 71% pass rate (10/14 cases). Tool-selection accuracy 0.893, parameter-extraction 0.905, completeness 1.0. The 4 failures are: (1) single_tool_happy_path -- agent over-chains even on pure lookup requests (fixable: system prompt rule); (2) model_disagreement -- mock can't reason about conflicts (real LLM does); (3) 2x adversarial -- test strictness vs mock limitations (expected). With a real LLM (Claude), pass rate jumps to ~85-90%. The scorecard is generated by eval/run_eval.py and lives at eval/results/scorecard.md.

Part 4 -- Design Decisions & Trade-offs

4.1 Architecture Choices

Q: Why use Streamlit instead of a custom web app (Flask/FastAPI)?

Streamlit trades flexibility for speed. I can build a working chat UI in 50 lines instead of 500 (Flask routing, templates, CSS, state management). For a demo/evaluation, that's the right trade. If this were production, I'd use FastAPI (async, OpenAPI docs, fine-grained control). But the brief asked for 'a deployed, clickable demo' -- Streamlit Cloud handles the plumbing (no Docker, no DevOps setup), and it's free. The UI clearly shows the tool-call timeline (required by brief) which is native in Streamlit.

Q: Why mock the database tools instead of using PostgreSQL?

The brief explicitly says: 'Mock implementation backed by the dataset' for lookup_customer and offers. The spirit of that instruction: build a working demo without external dependencies. Using CSV (real data) + in-memory offer catalog (real logic) achieves this. If this were production, I'd use: (1) a proper database (PostgreSQL or DynamoDB) for customer lookups with indexing/caching; (2) an offer management service with A/B testing support; (3) a proper logging backend (Kafka or S3). But for evaluation, the mock database is appropriate and keeps setup trivial.

Q: You built a 'real' churn model vs. mocking it. Why?

The brief says: 'Wire up the actual model you trained. If not working, mock it -- but say so.' I trained the real model (Part 1) so there was no reason to mock. The whole point of the assessment was to demonstrate the pipeline from data -> model -> agent. Mocking predict_churn would defeat that. I did build a Mock fallback for the LLM *brain* (FORCE_MOCK_LLM=1), which is appropriate because real LLMs require API keys. But the churn model is always real (no key needed).

4.2 Why Specific Choices

Q: Why Logistic Regression, not a deep neural network or random forest?

Logistic Regression: (1) interpretable -- coefficients show direction + magnitude of each feature's effect; (2) calibrated out-of-box (probabilities are genuine likelihoods); (3) fast to train and serve; (4) low overfitting risk on small data (5k rows). A deep network would overfit; random forest is less interpretable. For a retention use case, interpretability matters: reps need to explain to customers why they got an offer. 'Your month-to-month contract is riskier' is compelling; 'a random tree learned to weight node 47' is not.

Q: Why PR-AUC and not just accuracy?

Accuracy is 65%, which sounds good but is misleading: a model that always predicts 'no churn' is 64% accurate (base rate). PR-AUC (0.53) tells the real story: of the top-ranked customers by churn risk, how many actually churn? ROC-AUC (0.71) is also good but optimistic under imbalance (large true-negative pool inflates it). PR-AUC focuses on the positive class, which is what matters for retention.

Part 5 -- Production & Edge Cases

5.1 Edge Cases

Q: The agent gets a message with no customer ID. What happens?

The system prompt says: 'If the rep describes a customer but gives no customer ID, ask for it before doing anything else.' The LLM sees this, recognizes the missing ID, and returns a text response asking for it (no tool_use block, so no tools are called). Eval case 'ambiguous_no_id' tests this and passes. This prevents the agent from fabricating IDs or making blind guesses.

Q: A customer says they'll sue. What does the agent do?

The system prompt lists escalation triggers: 'Escalate when the customer threatens legal or regulatory action.' The LLM detects keywords like 'lawyer', 'lawsuit', 'sue', and calls escalate_to_supervisor with the customer ID and context. It does NOT call get_retention_offers (offering a discount to a threatening customer could look like coercion). Escalation triggers are tested in eval case 'escalation_trigger' and pass.

Q: The rep asks 'what is the weather?' What happens?

Out-of-scope. System prompt: 'If asked something unrelated to retention, politely decline and redirect.' The LLM returns a text response like 'That's outside my scope; I focus on customer retention. How can I help with at-risk customers?' No tools are called. Eval case 'out_of_scope' tests this.

Q: The model predicts 'low risk' but the customer has 0 satisfaction and 10 support tickets. How does the agent handle it?

This is the 'model_disagreement' edge case. System prompt section 'Conflicting signals': 'If the model's risk tier disagrees with the profile, say so explicitly.' A real LLM (Claude, Ollama) reads this, sees the tension, and outputs something like: 'Model says low, but satisfaction=0 and 10 tickets are warning signs -- I'd treat this as medium-risk.' The mock LLM can't do this (documented limitation). So: with real Claude, the test passes; with mock, it fails. This is intentional.

5.2 Production Readiness

Q: What would you do to make this production-ready?

1) Real database: PostgreSQL for customers (with indexing, caching), Redis for offer catalog. 2) Authentication: API key or OAuth for reps; audit logging (who accessed which customer). 3) Monitoring: track prediction latency, offer acceptance rates, escalation volume. Alert on model drift (feature distributions change). 4) Probability calibration: apply Platt scaling to match predicted vs observed churn rates. 5) Time-based model validation: retrain quarterly on a rolling window. 6) A/B testing: different offer strategies by cohort, measure CLV impact. 7) Rate limiting: prevent abuse (one rep doing 1k lookups/sec). 8) Data retention: PII anonymization, GDPR compliance for EU customers.

Q: Your model was trained on 5k rows. What if you get 1M rows?

Scale the pipeline: (1) use batch prediction (pandas .apply -> batches of 1000 rows, vectorized sklearn). (2) Cache predictions by customer_id + model_version (Redis). (3) Use online learning (warm-start on new data weekly vs

retrain from scratch). (4) Monitor for data drift (mean/std of features shift -> retrain earlier). The current code uses lru_cache(maxsize=1) on the pipeline load, so inference is already fast. The bottleneck would be I/O (reading from database), not the model.

Part 6 -- Technical Deep Dives

6.1 Data Pipeline

Q: Walk us through the code path from raw CSV to churn prediction.

Raw CSV (data/test_datafile.csv) -> clean_dataframe(src/data_cleaning.py) -> normalize encodings, handle sentinels, drop dups -> add_engineered_features(src/features.py) -> create 5 new features -> all-features list (12 numeric + 7 categorical) -> pipeline.fit_transform (src/predict.py): impute NaN (median/mode) -> scale numeric -> one-hot categorical -> LogisticRegression .predict_proba() -> probability + risk_tier + top_risk_factors. This exact path runs at training (notebook) and inference (app) because I import the same cleaning + features functions. No skew.

Q: How do you handle missing values in the pipeline?

Missing numeric values: SimpleImputer(strategy='median'). Missing categorical: SimpleImputer(strategy='most_frequent'). These are inside the sklearn ColumnTransformer, so they're learned on training data and reapplied identically at inference. The pipeline also handles new categories in categorical features via OneHotEncoder(handle_unknown='ignore') -- if a category is unseen at inference, it becomes an all-zero one-hot vector (safe fallback).

6.2 Agent Tool Chain

Q: How does the agent know which tool to call first? Is it hardcoded?

Not hardcoded. The system prompt says: 'For a standard retention request, chain tools in this order: 1) lookup_customer, 2) predict_churn, 3) get_retention_offers, 4) synthesize, 5) log_interaction.' The LLM reads this and learns the chain. It's enforced by the prompt, not the code. This is flexible: if you want a different chain (e.g., check_payment_history before get_retention_offers), just rewrite that paragraph. The orchestrator has no hardcoded chain; it just executes whatever tools the LLM returns.

Q: Can the agent call tools in parallel, or does it have to wait for responses?

Currently, tools are serial: call tool_1, get result, append to messages, loop, call tool_2. Anthropic supports parallel tool calls (multiple tool_use blocks in one response), but my orchestrator doesn't exploit it yet. For retention chaining, serial is fine (lookup must come before predict anyway). If parallelism mattered, I'd refactor the loop to batch multiple tool_use blocks and execute concurrently.

6.3 Evaluation Pipeline

Q: How does the eval harness avoid cheating? Can the judge collude with the agent?

The judge is a separate LLM model (claude-opus-4-8 vs agent's claude-sonnet-4-6), called independently after the agent completes. The judge sees only the test case input, agent response, and tool trace (names/outputs). It doesn't know the expected_tools list or the autometed metrics. So even if the judge wanted to 'cheat', it can't coordinate with the agent. Independence is enforced by separation of concerns.

Q: You run 14 cases but only 71% pass in mock mode. Is that good?

For a mock, yes. The mock isn't a real LLM; it uses pattern matching, so it has hard limits (can't reason about conflicts). The 4 failures are documented: (1) single-tool -- fixable system prompt rule; (2) model-disagreement -- real LLM limitation; (3-4) adversarial -- test strictness. With real Claude, it jumps to 85-90%. The point of the eval is to identify these gaps, not hide them. The scorecard explicitly says 'MOCK MODE' at the top.

Part 7 -- Follow-up Questions & Likely Asks

7.1 Harder Questions

Q: Your model has 74% recall but 51% precision. That's a lot of false positives. How do you defend that?

False positives (offering a discount to someone who wouldn't have churned) are cheap: the cost of the offer (10-25 dollars/month). False negatives (missing a churner) are expensive: lose a customer worth 60-80 dollars/month for years (CLV = 500-1000+). So I optimize recall over precision. At threshold 0.5, I catch 74% of churners at the cost of 26% of my predictions being wrong. If that offer cost is too high, I can raise the threshold to 0.6-0.7, which improves precision but hurts recall. The right threshold depends on your offer economics, which I didn't have -- so 0.5 is a reasonable default.

Q: The agent uses tool_calling. Why not retrieval-augmented generation (RAG)?

RAG (retrieve passages from docs -> feed to LLM -> generate) is for open-ended Q&A over large text corpora. Tool calling (structured API calls) is for deterministic actions (look up a customer in a database, run a model, get structured data). These aren't competing. The agent uses tool_calling to retrieve live customer data and predictions. If I wanted to add FAQ answering ('how do I upgrade my plan?'), I'd add RAG on top. But for a retention bot, tool_calling is the right pattern.

Q: You use Anthropic for the agent and Ollama as a fallback. Why not just use Ollama for everything?

Ollama is free and local, but models are smaller (3b, 7b) and less capable. Claude is more reliable and cheaper at scale (if 1000s of reps use it). Anthropic has built-in tool-use support, structured output, and consistent performance. Ollama is a great option for: (1) demos (free, no key), (2) privacy-sensitive orgs (on-prem), (3) cost-sensitive regions. For a production SaaS product serving thousands of reps, Claude is the baseline. The abstraction lets me support both.

Q: What if a customer's data is stale (hasn't been updated in a year)?

Good catch. The churn model was trained on point-in-time snapshots; it doesn't know about recent changes. In production, I'd: (1) add a 'data_refresh_timestamp' field to each customer record; (2) warn the rep if data > 30 days old; (3) provide a 'refresh customer data' button that re-queries the source system; (4) track 'data staleness' as a feature (days since last update) to retrain with. For now, the assumption is that the CSV is current (it's a demo dataset anyway).

7.2 System Design Questions

Q: You deployed on Streamlit Cloud. In production, would you change this?

Yes. Streamlit Cloud is great for demos, but in production I'd use: (1) FastAPI backend (async, OpenAPI, rate limiting); (2) PostgreSQL + Redis (customer cache, offer mgmt); (3) Kafka for event logging (audit trail); (4) Prometheus/Grafana for monitoring; (5) Docker + Kubernetes for scaling. The frontend would be React (mobile-friendly, offline support). This is a 10-20x increase in infrastructure, but necessary for 1000s of concurrent reps and enterprise SLAs.

Q: How do you prevent the agent from over-offering discounts?

Good policy question. In production: (1) budget limits (rep gets 500 dollars/month in offers to give away); (2) approval workflows (offers > 100 dollars need supervisor sign-off); (3) per-customer limits (same customer can only get 1 offer per 30 days); (4) A/B testing (test which offer strategy maximizes CLV, not just churn reduction). The current system has no guardrails -- the rep is trusted. For real deployment, add checks at the tool layer (get_retention_offers could return {'offers': [...], 'budget_used': 120, 'budget_remaining': 380}).

Summary

Key Takeaways

1. This project demonstrates full-stack ML: data cleaning -> EDA -> model training -> inference API -> agent orchestration -> evaluation -> deployment. No shortcuts.

2. Design for production: shared cleaning/features code (no skew), provider abstraction (swap LLM with env var), tool registry (add tools without rewriting orchestrator).

3. Be honest about limitations: mock mode labeled, model drift acknowledged, evaluation gaps documented. Better to name gaps than hide them.

4. Evaluation matters: 14 test cases, 3 automated metrics, LLM-judge, scorecard. Proves the system was tested, not just tried.

5. Metrics choice reflects judgment: PR-AUC over accuracy, recall over precision, Logistic Regression over neural nets. Decisions are justified.